Exam 2: Exploring Data – Version D

Exam Agreement

This exam is an **individual assessment** and you should neither offer help to nor receive help from another person.

Cell phones are to be handed in to the instructor and all other electronic devices must be stored in bags at the front of the room.

No smart glasses or other wearable electronic devices are to be powered on during this exam except with accommodations.

You may use **no physical or electronic** resource to complete the work.

By signing, you agree that you have neither given nor received unauthorized aid on this examination.

Printed Name: _____

Chapman ID: _____

Signed: _____

Section: _____ **Date:** _____

Include this signed page as the first page of your submitted work.

Exam Instructions

You must answer every question on the exam to the best of your ability. Include all appropriate code, syntax, and functions that would lead to the code to run successfully.

Question 1: Loading Data

Your data file "balanus-data.csv" contains your measurements of a population survey of Pacific acorn barnacles (*Balanus glandula*) which you measured on the pilings of docks in Newport Harbor. The measurements contain total width of each barnacle in the survey area and the date the survey was collected. The first line contain information about the collection site and other notes, and the second line has column names. Write code that will read this CSV into R.

Question 2: Cleaning Data

Originally saved from Excel, your data set from Question 1 now contains a lot of empty rows and columns. There are also some missing observations, where the initial value was removed. Using the tidy pipe operator, clean the data for three things: standardize the column names, remove the empty rows and columns, and remove every observation that is incomplete.

Question 3: Basic Data Analysis

Every year, Newport Harbor has to pay to have the dock pilings scraped to prevent the buildup of fouling organisms like your barnacle. Recently, they have invested a polymer coating for the pilings that supposedly makes it harder for barnacles to continue to stick to the piling after they get to be a certain size. The harbor has asked you to plot your data to show them whether or not the coating is working. What plot would you use to answer their question and why? Write the code for the plot using either base graphics or the `ggplot2` package.

Question 4: Troubleshooting Compound Code

You are attempting to load the tomato haul data set with the following code:

```
tomatoes <- read.csv("tomatohaul2021.csv")
```

But you get the error:

```
Error in read.table(file = file, header = header, sep = sep, quote = quote,  :
  more columns than column names
```

What does this error mean, and how do you fix it?

Question 5: Plotting with Base Graphics

Using base graphics, create two box plots of the `mtcars` data set (Fig. 1), each one showing `wt` versus `gear` one of the values of `vs` (either 0 or 1).

```
mtcars          32 obs. of 11 variables
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
 $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

Figure 1: The mtcars data set.

Question 6: Plotting with ggplot2

Using `ggplot2` and the `mtcars` data set (Fig. 1), create one box plot showing `wt` versus `gear` for both values of `vs` as filled-color boxes.

Question 7: How We See

False-color illusions work by staring at a color and then quickly transitioning to a neural background. This happens as a result of visual adaption over time. Explain how this works at a cellular level.

Question 8: Visual Processing and Attention

Describe a general way that you can reduce the demand on a viewer's visual working memory. Provide an example that demonstrates this principle.

List of Functions for Exam 2

- `+, -, *, /, ^` – arithmetic operators
- `sqrt()` – square root
- `log()` – logarithm
- `seq()` – sequence creation
- `rep()` – value/vector repetition
- `sort()` – vector sort
- `length()` – determine vector length
- `[ ]` – vector subset/extraction, list slicing, multiple members
- `sum()` – sum all vector elements
- `prod()` – multiply all vector elements
- `mean()` – averages all vector elements
- `sd()` – takes standard deviation of all vector elements
- `matrix()` – create a matrix
- `rbind()` – create a matrix (bind rows)
- `cbind()` – create a matrix (bind columns)
- `dim()` – get matrix dimensions
- `nrow()` – get number of rows
- `ncol()` – get number of columns
- `[ , ]` – matrix/array/data frame subsetting
- `diag()` – diagonal elements/identity matrix
- `array()` – create an array
- `list()` – create a list
- `[[ ]]` – unnamed member reference list
- `$` – get named member/variable
- `data.frame()` – create data frames
- `==, !=, >, <, >=, <=` – relational operators
- `any()` – checks whether any entries are true
- `all()` – checks whether all entries are true
- `&&, &, ||, |, !` – logical operators
- `which()` – determines indices of `TRUE`s
- `nchar()` – gets the number of characters in a string
- `cat()` – concatenates strings (no return)
- `paste()` – pastes strings (returns a string)
- `\` – escape character
- `substr()` – Subsets a string
- `sub()`, `gsub()` – string matching and replacement
- `factor()` – creates a factor vector
- `levels()` – gets levels of a factor
- `cut()` – creates factor from continuum
- `is.infinite()` – element-wise check for `Inf`
- `is.finite()` – element-wise check for finiteness
- `NaN` – value for invalid numerics
- `is.nan()` – element-wise check for `NaN`
- `NA` – value for missing observation
- `is.na()` – element-wise check for `NA` or `NaN`
- `na.omit()` – delete all `NA`s and `NaN`s
- `NULL` – value for empty
- `is.null()` – check for `NULL`
- `attributes()` – list explicit attributes
- `attr()` – obtain specific attribute
- `dimnames()` – get array dimension names
- `class()` – get object class
- `is._()` – object-checking functions
- `as._()` – object-coercion functions
- `unlist()` – object-coercion from list to vector
- `runif()` – draws pseudo-random numbers from a uniform distribution
- `sample()` – draws numbers from a sample given by user

- `plot()` – makes a plot in base graphics
- `~` – creates formula object
- `points()` – adds points to a plot in base graphics
- `lines()` – adds points to a plot in base graphics
- `legend()` – adds a legend to a plot in base graphics
- `pivot_wider()` – restructure data to wide format (tidyr)
- `pivot_longer()` – restructure data to long format (tidyr)
- `ggplot()` – initiates a plot using ggplot2
- `geom_x()` – geometric object layers in ggplot2
- `ggtitle()` – title a ggplot
- `labs(), xlab(), ylab()` – alter axis labels in ggplot2
- `lims(), xlim(), ylim()` – alter axis limits in ggplot2
- `summary()` – produces summary stats on a data frame
- `head()` – prints first lines of a data frame
- `tail()` – prints last lines of a data frame
- `IQR()` – computes interquartile ranges
- `median()` – computes median
- `table()` – builds a contingency table of counts
- `quantile()` – produces sample quantiles
- `tapply()` – apply function to groupings in data set
- `apply()` – apply function to row or column of data set
- `read.table()` – imports generic text file
- `read.csv()` – imports comma separated value file
- `clean_names()` – standardizes column names in data frame (janitor)
- `remove_empty()` – removes empty columns and/or rows in data frame (janitor)
- `%>%` – tidy pipe operator (tidyr/janitor)
- `write.csv()` – exports object to text csv file